

크래프톤 정글 12기 · 303호 4팀

AI 기반 Kubernetes 장애 분석 및 복구 자동화 서비스

크래프톤 정글 12기
303호 4팀

AI 기반 Kubernetes 장애 분석 및 복구 자동화 서비스

주간 공유 / 멘토링 보고서
2026-06-29

팀원: 임가인, 이민정, 전우현, 정찬빈, 최우녕
멘토: 김시훈 멘토님, Krafton Data Engineering Dept.

목차

1. 프로젝트 개요
2. 문제 정의와 선정 배경
3. 제품 범위와 하지 않는 일
4. 팀 구성 및 역할
5. 수행 절차와 WBS
6. 멘토링 피드백과 반영 이력
7. 현재 아키텍처
8. Event / Queue 시스템 진행
9. Gateway / Auth 진행
10. GitOps / Command 진행
11. Target / Telemetry / Evidence 진행
12. RCA / Safe PR / Audit 진행
13. 브랜치 / PR / 이슈 현황
14. 위험요소와 대응 계획
15. 다음 주 실행 계획
16. 자체 평가
17. Appendix. 근거 문서와 멘토링 일지

프로젝트 한 문장 정의

Kubernetes 장애가 발생했을 때 Git 변경, 클러스터 증거, AI RCA, Safe PR, 대시보드 상태를 이벤트 기반으로 연결하는 운영 보조 도구다.

사용자 필요한 것

Kubernetes 운영자 어떤 변경이 장애와 연결되는지 빠르게 보고, 안전한 복구 경로를 확인한다.

개발자 내가 만든 변경이 장애와 관련 있는지 근거를 보고, 수정 PR 초안을 얻는다.

팀 리더 운영 이벤트, RCA, command, audit 흐름을 한 곳에서 추적한다.

해결하려는 문제

문제 왜 중요한가

Git, Kubernetes Event, Pod 상태, Log, Metric이 흩어져 있음 장애 원인을 찾기 위해 여러 도구를 오가야 하고, 초보자는 어떤 순서로 봐야 하는지 모른다.

긴급 수동 조치가 GitOps desired state와 어긋날 수 있음 kubectl로 급히 고친 내용이 Git repo 상태와 달라져 다음 배포 때 다시 깨질 수 있다.

AI가 설명만 하면 운영에 바로 쓰기 어려움 근거, 변경 제안, 검증 방법, rollback 계획까지 연결되어야 실무에 쓸 수 있다.

타깃 클러스터에 분석 시스템을 직접 붙이면 운영 난도가 커짐 target cluster는 보호되어야 하므로 agent는 outbound로만 붙고, 관리 기능은 management cluster에 둔다.

프로젝트가 하는 일과 하지 않는 일

[하는 일]

session/login

- Git polling 또는 UI command
- NATS JetStream event
- GitOps / Command / RCA / Dashboard / Audit worker
- Target Cluster Agent
- evidence/result event
- Dashboard read model
- Safe PR event

[하지 않는 일]

제외 범위 이유

AI가 승인 없이 production namespace를 직접 수정 안전성과 감사 가능성 문제
MVP에서 모든 외부 SaaS/API를 real integration으로 강제 2주 MVP 기간에 통합 실패 위험이 큼
UI가 DB나 JetStream에 직접 접근 Gateway가 외부 HTTP 경계를 책임져야 함
Target Agent가 관리 NATS에 직접 접속 target cluster 보안 경계와 outbound-only 원칙 유지
secret/token/kubeconfig를 event/log/response에 포함 보안 사고 방지

프로젝트 특화 포인트

특화 포인트 설명

Event-driven operation flow command, evidence, RCA, PR, dashboard, audit을 event subject와 correlation_id로 연결

Evidence-first RCA raw log/metric 전체가 아니라 장애 판단에 필요한 증거 묶음을 만든 뒤 RCA 수행

Safe PR 중심 복구 자동 변경 대신 GitHub PR로 변경안, 근거, 검증 방법, rollback을 남김

Sandbox guard production write는 금지하고 sandbox namespace부터 제한 실행

DLQ/replay 운영성 worker 실패를 retry 후 DLQ로 보내고 운영자가 재처리 가능

Outbound target agent target cluster가 management로 outbound 연결만 수행해 보안 경계를 단순화

팀 구성 및 역할

구성원 역할 담당 범위 이번 주 상태

임가인 RCA / Safe PR / Audit `services/rca-worker`, `services/gitops/repo-gateway-worker`, audit/projection 연계 RCA 이벤트 체인과 repo-gateway 위임 기준 문서화, 이슈 #505~#507

이민정 Target / Telemetry `services/target/target-cluster-agent`, `services/target/node-collector`, `deploy/target` Prometheus 폐쇄 루프와 Evidence 학습 task #527~#537 분해

전우현 GitOps / Command Git polling, manifest render, desired diff, command worker polling-first GitOps, command state/policy 이슈 #503~#504

정찬빈 Gateway / Auth `services/api-gateway`, session, RBAC, integration/credential email/password + Redis session 브랜치 진행, #519~#526 분해

최우녕 Platform / Integration event runtime, contracts, DLQ/replay, WBS/docs/WIKI, 통합 event-system #502, docs/PR/WBS 정리

김시훈 멘토님 멘토 MVP 범위, 실무 관점, 학습 방향, trade-off 2026-06-26 멘토링, 온보딩 자료 공유

수행 절차와 전체 WBS

기간 마일스톤 데모 목표 주 담당

2026-07-01 1차 보고 Event Runtime / DLQ / Node Collector Platform/Integration

2026-07-08 2차 보고 GitHub OAuth + Safe PR 초안 Gateway/Auth, RCA/Safe PR

2026-07-15 3차 보고 실제 Telemetry Evidence Target/Telemetry, RCA/Safe PR

2026-07-22 4차 보고 GitOps command 운영 플로우 GitOps/Command, Platform/Integration

2026-07-29 최종 보고 장애 주입부터 RCA, Safe PR, 복구 제안까지 end-to-end 전체

[WBS 요약]

항목 값

전체 작업 56

P0 작업 44

완료 0

진행 중 1

미시작 55

2026-07-01 1차 데모 상세

첫 번째 수요일 데모는 기반 설명이 아니라 실제 흐름을 한 번 끝까지 연결하는 데 집중한다.

순서 데모 항목 검증 명령 또는 화면

- 1 management/target 클러스터 기동 `make up`, `make status`
- 2 GitHub webhook 또는 수동 command 입력 `POST /github/webhook`, `POST /commands`
- 3 event bus와 worker 처리 확인 NATS event, `event\_processing`, worker log
- 4 Target Agent command polling/result 확인 `/agent/commands/poll`, `/agent/commands/{id}/result`
- 5 telemetry evidence와 RCA baseline 확인 `/agent/evidence`, `evidence.built`, `rca.completed`
- 6 Safe PR dry-run/fake 확인 `safe\_pr.created`, fake PR record
- 7 dashboard read model/query/stream 확인 `GET /dashboard/query`, `GET /dashboard/stream`
- 8 DLQ 생성/조회/replay 확인 `GET /dead-letters`, `POST /dead-letters/{id}/replay`

멘토링 피드백 요약

2026-06-26 멘토링 핵심 요약:

피드백 의미

시스템을 크게 만드는 것보다 한 사이클이 실제로 돌아가는 MVP가 우선 모든 구성요소를 현업 수준으로 완성하려 하지 말고 vertical slice를 먼저 연결
OAuth/auth는 너무 커질 수 있음 인증 자체가 프로젝트가 되지 않도록 session과 GitHub 연동 수준으로 시작
PostgreSQL과 Prometheus/metric store 중심으로 단순화 모든 데이터를 DB/Redis에 넣지 말고 데이터 성격에 맞는 저장소 선택
AI가 어떤 운영 인사이트를 주는지가 핵심 metric/log 수집 자체보다 RCA와 다음 행동 설명이 중요
secret/token은 Git/event/log에 넣지 않기 AI 작업 중 secret 노출 사고 방지
질문은 구체적인 trade-off로 모으기 전체 코드 리뷰보다 기능 단위 설계 선택지를 질문

멘토링 반영 내역

멘토 피드백 반영 결과 근거 문서/이슈

MVP 한 사이클 우선 7/1 데모를 Event Runtime, DLQ, Node Collector 중심으로 재정의 WIKI WBS, WBS xlsx

OAuth/auth 범위 축소 OAuth-only 폐기, 내부 로그인 + Redis session + integration credential 구조로 재설계 `docs/team/member-guides/gateway-auth.md`, #519~#526

저장소 단순화 PostgreSQL은 구조화 데이터, Prometheus는 metric, log는 Loki/object류로 분리 WIKI 멘토링 회의록, `docs/architecture.md`

AI 인사이트 강조 Evidence → RCA → Safe PR/Command 흐름 중심으로 정리 #505, Target/Telemetry 문서

secret 노출 금지 event payload에는 token이 아니라 ref/id만 전달 `docs/events.md`, #501, #502

팀원 지식 차이 보완 member guide와 micro task 문서로 작업 단위를 세분화 Gateway/Auth guide, Target/Telemetry 하위 문서

현재 아키텍처 핵심 결정

결정 이유

management cluster와 target cluster 분리 분석/관리 시스템과 사용자 workload를 분리해 안전성 확보
처음부터 서비스별 `app.py` entrypoint 단일 FastAPI app/role dispatcher로 회귀하지 않기 위해
Gateway 하나만 외부 HTTP 경계 auth, policy, input validation, event publish 순서를 중앙에서 통제
내부 비동기 흐름은 NATS JetStream worker 장애, retry, DLQ, replay, event audit이 필요
Target Agent는 outbound-only target cluster를 외부에서 직접 열지 않기 위해
sandbox namespace만 write production write 사고 방지
UI는 Gateway API만 호출 DB/JetStream 직접 접근 금지

Event / Queue 시스템 진행

항목 현재 기준

Stream NATS JetStream `SERVICE\_EVENTS`

Envelope `event\_id`, `subject`, `source`, `correlation\_id`, `causation\_id`, `created\_at`, `payload`

Delivery at-least-once delivery

Consumer durable pull consumer

Idempotency `event\_processing(event\_id, consumer)` ledger

Retry bounded retry

DLQ max retry 초과 시 `event\_dead\_letters` 저장, `dead\_letter.created` 발행

Replay Gateway DLQ API에서 원본 이벤트 재발행

Outbox 업무 write와 event publish 불일치 방지를 위한 hardening 구조

[현재 구현 증거]

- `packages/runtime/app.py`: `App`, `@app.sub(...)` , `@app.on\_event(...)`
- `packages/runtime/worker.py`: 공통 ack/retry/DLQ 처리
- `packages/contracts/event\_bus/bodies/`: 타입 있는 이벤트 body
- `packages/events/bus.py`: `NatsEventBus`, recorded publish
- `packages/storage/database.py`: events, event_processing, dead letter, outbox 저장 경계
- 테스트 증거: `uv run pytest -q` 42 passed, `make check` 통과

[남은 확인]

- `make up` 이후 `make smoke` 통과
- 최신 HEAD 기준 GitHub Actions CI 증거
- DLQ 조회/replay API 실제 Gateway 경로 최종 확인

Event System 리스크와 판단

리스크 현재 상태 대응

exactly-once 오해 현재 목표는 exactly-once가 아니라 at-least-once + idempotency `event\_processing` ledger와 stable id 사용
worker async 루프에서 sync DB 호출 handler 경로는 AsyncDb proxy로 비차단 경계 정리 실DB smoke 이후 남은 sync 경로 단계적 개선
worker liveness 확인 어려움 HTTP healthz 대신 heartbeat file + exec probe 설계 포트 노출 없이 kubelet이 stale heartbeat 감지
업무 write와 event publish 불일치 Outbox/UoW 구조를 hardening 후보로 정리 side effect 강한 작업부터 적용
smoke 실패 Gateway가 `localhost:18080`에서 실행되지 않아 실패 make up 환경에서 재검증 필요

Gateway / Auth 진행

단계 목표 상태

- #519 인증 계약과 route skeleton open
- #520 내부 로그인과 Redis session 진행 중, 브랜치 구현 증거 있음
- #521 Organization/Project 권한 모델 open
- #522 기존 Gateway API 권한 guard 적용 open
- #523 GitHub integration target과 credential ref open
- #524 Credential binding과 action 권한 open
- #525 Token Broker port와 fake vault open
- #526 Prometheus target으로 추상화 검증 open

[현재 구현 증거]

- 브랜치: `origin/feat/jcbbbbbb/api-gateway`
- 최신 커밋: `be7a2ab feat: 이메일/비밀번호 로그인 및 Redis 세션 추가`
- 변경 파일: `services/api-gateway/auth.py`, `gateway.py`, `settings.py`, `packages/contracts/gateway/requests.py`, `tests/test\_password\_auth.py` 등

[주의점]

- open PR 없음
- 최신 커밋 기준 CI 증거 없음
- 브랜치가 `dev` 보다 46 commits behind로 확인됨

Gateway / Auth 설계 판단

[왜 GitHub OAuth-only가 아닌가]

이유 설명

GitHub 계정은 우리 서비스 계정이 아님 우리 서비스의 user, org, project, role은 우리 DB가 소유해야 함
외부 도구는 GitHub만이 아님 Prometheus, Loki, Grafana, DB, Kubernetes, OTel 등은 각자 credential이 다른
권한 회수가 중요 command, DLQ replay, Safe PR은 즉시 세션 종료/권한 변경 반영이 필요
secret boundary 필요 worker가 provider token을 직접 들고 다니면 event/log 유출 위험

[단계별 최종 구조]

Identity: 우리 서비스 사용자/조직/팀/역할

Authorization: project, target, action 권한

Integration: GitHub, Prometheus, Loki, Grafana, DB, OTel

Credential Vault: 실제 secret 저장

Token Broker: policy 통과 후 짧게 credential 제공

Event System: secret 없이 id/ref/action만 전달

GitOps / Command 진행

[GitOps 흐름]

git polling 또는 git.changed
→ repo metadata 저장
→ manifest render
→ manifest.rendered
→ desired.diff.detected
→ diff.analyzed
→ command.requested 또는 safe_pr.requested

[Command 흐름]

command.requested
→ policy 검증
→ command.rejected 또는 command.dispatch.ready
→ agent command queue 저장
→ target agent polling
→ command.completed

[이슈 상태]

이슈 내용 상태

#503 Git 변경을 desired state 이벤트로 변환 open

#504 정책 검증과 target command 전달 open

[주요 설계 결정]

- GitOps worker는 GitHub에 직접 PR을 만들지 않는다.
- GitOps 책임은 repo 변경을 내부 desired state 이벤트로 바꾸는 것이다.
- 실제 클러스터 적용은 Command Worker / Target Agent 경계에서 처리한다.
- Command Worker는 명령을 바로 실행하지 않고 policy, 상태, queue를 관리한다.

Command 상태와 정책

Command Worker가 고민해야 하는 상태:

상태 의미

requested 사용자가 command를 요청했거나 diff가 command 후보를 만들었다.

rejected namespace/action/role/target 정책에서 거절됐다.

queued target agent가 가져갈 수 있게 queue에 저장됐다.

dispatched 어떤 route/channel/cluster로 보낼지 결정됐다.

acknowledged agent가 command를 lease 또는 수신했다.

succeeded agent 실행 결과가 성공이다.

failed agent 실행 결과가 실패 또는 timeout이다.

정책 rule 후보:

- namespace가 `sandbox` 인지
- action이 허용 목록에 있는지
- target cluster가 등록된 cluster인지
- 요청 user가 project role을 갖는지
- agent가 online인지
- 같은 command가 중복 전달되어도 idempotent한지

Target / Telemetry 진행

현재 정리된 방향:

- node-collector /metrics
- real Prometheus scrape
- Prometheus query API
- Python query client
- Agent debug query API
- MetricEvidence summary

[이슈 상태]

범위 이슈

- Prometheus 설치/검증 #527, #528
- Node Collector metric #529
- Prometheus scrape/query #530, #531
- Agent debug query API #532
- MetricEvidence summary #533
- Kubernetes pod/event reader #534
- K8s + Prometheus evidence 결합 #535
- Gateway 계약 연결 준비 #536
- Evidence 학습 task와 micro demo #537

Telemetry와 Evidence 사고방식

용어 쉽게 말하면 우리 프로젝트에서의 의미

raw metric Prometheus에 쌓인 숫자 시계열 CPU, memory, restart count, request rate 등

raw log Loki나 stdout에 있는 긴 로그 에러 메시지, stack trace, image pull 실패 로그 등

raw event Kubernetes Event Scheduling 실패, CrashLoopBackOff, BackOff, Pull 실패 등

Evidence RCA에 필요한 요약된 근거 어떤 pod가 언제부터 몇 번 재시작했고 어떤 로그/메트릭이 함께 변했는지

[Pull / Ingest 차이]

도구 기본 모델 의미

Prometheus pull / scrape Prometheus가 `/metrics` endpoint를 주기적으로 읽어간다.

Loki ingest / push log collector가 Loki로 로그를 밀어 넣는다.

OpenTelemetry ingest / export app 또는 collector가 trace/metric/log를 OTLP로 보낸다.

RCA / Safe PR / Audit 진행

RCA Worker 목표:

- cluster.evidence.received
 - evidence pack 정규화
 - rule 기반 baseline RCA
- evidence.built
- rca.completed
- safe_pr.requested
- repo-gateway-worker
- safe_pr.created 또는 safe_pr.failed

[현재 구현/계약 기준]

- 서비스 진입점: `services/rca-worker/app.py`
- Safe PR outbound gateway: `services/gitops/repo-gateway-worker/app.py`
- body 계약: `packages/contracts/event\_bus/bodies/rca.py`, `repo.py`
- 테스트 후보: `tests/test\_rca\_evidence.py`, `tests/test\_repo\_gateway\_worker.py`, `tests/test\_event\_golden\_path.py`

[중요한 결정]

- PR 생성은 RCA Worker가 직접 수행하지 않는다.
- RCA Worker는 `safe\_pr.requested`를 발행하고, repo-gateway-worker가 외부 GitHub 작업을 담당한다.
- AI가 없어도 rule 기반 최소 RCA가 나와야 한다.
- RCA 결과에는 원인, 근거, confidence 또는 판단 근거, suggested action이 포함되어야 한다.

[현재 리스크]

Dashboard / Audit Read Model

서비스 역할

Dashboard Projection Service 주요 event를 dashboard 화면용 read model로 변환

Audit Timeline Service 누가 어떤 event와 command, RCA, PR 흐름을 만들었는지 기록

현재 방향:

- UI는 아직 이번 5인 분배에서 제외하고, dashboard read model을 공통 영역으로 유지한다.
- UI가 DB나 JetStream에 직접 붙지 않고 Gateway query/stream API만 호출한다.
- 모든 주요 event는 correlation_id로 묶어 하나의 사건 흐름으로 볼 수 있어야 한다.
- DLQ/replay, command 상태, RCA 결과, Safe PR 결과도 dashboard/audit에서 추적 가능해야 한다.

브랜치 / PR 현황

구분 최신 상태 의미

`origin/main` `5b0b399`, PR #541 merge GitOps polling API 설계 문서 main 반영

`origin/dev` `c025a85`, PR #540 merge GitOps polling API 설계 문서 dev 반영

`origin/feat/woonyong-kr/event-system` `6018211` crash test/rollout wait fix 포함

`origin/feat/jcbbbbbb/api-gateway` `be7a2ab` email/password + Redis session 구현

`origin/feat/minmings111/target-cluster-agent` `ec0d1f9` 확인, 이슈에는 더 최신 구현 증거 언급 pod logs with Alloy, target telemetry 진행

`origin/feat/jeonwoohyun-hydromel/\*` 초기화 상태 GitOps/Command 구현 필요

`origin/feat/ummfieg/\*` 초기화 상태 RCA/Audit/Projection 원격 구현 diff 필요

[PR 현황]

PR base 내용 상태

#538 dev Evidence 학습 가이드와 데모 추가 merged

#539 main Evidence 학습 가이드와 데모 추가 merged

#540 dev GitOps polling API 설계 정리 merged

#541 main GitOps polling API 설계 정리 merged

GitHub Issue 현황

이슈 담당 내용 상태 판단

- #501 정찬빈 API Gateway 인증/요청 경계와 이벤트 발행 흐름 진행, 하위 #519~#526
- #502 최우녕 Event System: JetStream/DLQ/retry/outbox 진행, local 검증 증거 있음
- #503 전우현 GitOps Sync Worker open, 구현 필요
- #504 전우현 Command Worker open, 구현 필요
- #505 임가인 RCA Worker source 구현 증거 있으나 담당 원격 브랜치 반영 필요
- #506 임가인 Dashboard Projection open
- #507 임가인 Audit Timeline open
- #508 이민정 Target Cluster Agent 진행, 경로 불일치 정리 필요
- #509 이민정 Node Collector open
- #519~#526 정찬빈 Gateway/Auth 세부 task open
- #527~#537 이민정 Target/Telemetry 세부 task open

주요 리스크

리스크 가능성 영향 대응

OAuth/GitHub 실제 API 지연 높음 중간 fake adapter 유지, GitHub client feature flag

팀원별 telemetry 환경 차이 높음 높음 fake adapter와 Node Collector fallback 유지

event 중복 처리 중간 높음 `event\_processing` ledger, idempotent write, outbox relay 후보

수요일 데모 직전 통합 실패 중간 높음 화요일 저녁 integration freeze, 수요일 오전 critical fix만 허용

UI 지연 중간 중간 Gateway API와 curl/httpie demo 백업

RBAC 권한 과다 낮음 높음 sandbox write만 허용, `kubectl auth can-i` 확인

실제 PR 생성 사고 낮음 높음 feature flag, dry-run mode, test repo 우선

문서와 코드 불일치 중간 중간 PR마다 source docs/WIKI 갱신 체크

브랜치 behind/경로 불일치 높음 높음 PR 전 dev rebase/merge, 현재 서비스 경로로 재정렬

이번 주 성과

영역 성과

제품/기획 Kubernetes 운영 보조 도구로 문제/사용자/하지 않는 일 재정의

멘토링 2026-06-26 멘토링 회의록과 자동 전사 정리, 피드백 반영

WBS 56개 작업, 5개 담당 영역, 수요일 데모 마일스톤 정리

Event System envelope, body, subject, retry, DLQ, replay, outbox 경계 문서/구현 진전

Gateway/Auth OAuth-only 탈피, 내부 로그인/session/RBAC/integration/token broker 단계 설계

Target/Telemetry Prometheus 폐쇄 루프와 Evidence 학습 task를 세부 이슈로 분해

GitOps polling-first API 설계 문서 PR #540/#541 merge

Evidence 문서 Evidence 학습 가이드와 데모 문서 PR #538/#539 merge

이번 주 아쉬운 점

아쉬운 점 영향 보완

구현 브랜치 일부가 dev보다 많이 뒤처짐 PR 충돌과 경로 불일치 위험 다음 작업 전 rebase/merge 정렬

실제 smoke 검증이 완전히 통과하지 않음 demo 신뢰도 부족 `make up && make smoke && make status` 재검증

WBS상 완료가 아직 0개 진행률을 설명하기 어려움 작게 쪼갬 이슈부터 완료 후보 생성

문서량이 많아 팀원이 부담을 느낄 수 있음 초보자 onboarding 난이도 상승 member guide를 “읽는 순서”와 “첫 task” 중심으로 유지

실제 provider 연동이 아직 약함 최종 제품성 약화 fake 유지하되 GitHub/Prometheus 하나씩 real adapter 전환

다음 주 실행 계획

[2026-06-29 ~ 2026-07-01 우선순위]

우선순위 작업 담당

- 1 `make up && make smoke && make status` 실제 통합 검증 Platform/Integration
- 2 DLQ 생성/조회/replay 데모 경로 고정 Platform/Integration
- 3 Gateway/Auth 브랜치 dev 기준 재정렬과 PR 준비 Gateway/Auth
- 4 Target/Telemetry 경로 불일치 정리 Target/Telemetry
- 5 Node Collector `/metrics`와 Prometheus scrape/query 최소 확인 Target/Telemetry
- 6 Git polling → manifest render → diff event skeleton 시작 GitOps/Command
- 7 command state/policy 테스트 시작 GitOps/Command
- 8 RCA baseline fixture와 Safe PR requested 흐름 원격 브랜치 반영 RCA/Safe PR

[수요일 보고 전 freeze]

- 화요일 저녁까지 demo branch 고정
- 수요일 오전에는 critical fix만 허용
- UI가 없으면 curl/httpie로 demo
- 실패 시 fake adapter fallback 사용

자체 평가

평가 항목 내용

기획 의도 부합 운영 데이터와 GitOps/Command/RCA/Safe PR을 연결하는 목표는 멘토링 피드백 이후 더 명확해졌다.

실무 활용 가능성 production 자동 복구 대신 evidence, approval, Safe PR, audit을 강조해 실무 안전성과 설명 가능성을 확보하는 방향이다.

기술 난이도 event-driven microservices, Kubernetes telemetry, credential boundary, outbox/DLQ 등 난도가 높다.

현재 완성도 설계/문서/WBS/이슈 분해는 많이 진행됐으나, 실제 통합 데모와 구현 PR은 아직 부족하다.

다음 개선 smoke 통과, PR 단위 merge, 팀원별 첫 성공 경험, provider 하나씩 real adapter 전환이 필요하다.

팀원별 다음 액션

팀원 다음 액션 완료 증거

임가인 RCA fixture 기반 baseline, `safe\_pr.requested` 흐름을 담당 브랜치에 반영 `tests/test\_rca\_evidence.py`, repo-gateway test, PR

이민정 Node Collector `/metrics`, Prometheus scrape/query, 경로 정리 target 경로 정합성, query client test, demo log

전우현 Git polling metadata, manifest render skeleton, command policy test `git.changed` fixture test, command rejected/queued test

정찬빈 login/session 브랜치 재정렬, protected route guard, org/project 모델 시작 auth test, 401/403 test, dev base PR

최우녕 event-system smoke, DLQ/replay demo, docs/WIKI/issue 정합성 make check/smoke log, DLQ replay screenshot/log

Appendix - 멘토링 일지

일시 방법 내용 조치

2026-06-26 09:26 이메일 김시훈 멘토님 Teams 미팅 초대. MVP 기획, WBS, 각자 역할/목표 준비 요청 팀원에게 링크 공유, 미팅 준비

2026-06-26 11:04 이메일 답변 MVP 기획/WBS/역할/목표와 공부 방향/실무 질문을 준비하겠다고 답변 팀원에게 멘토님 요청사항 공유

2026-06-26 19:49 이메일 기존 기획 PPT를 멘토링 전 공유 멘토링 참고 자료로 사용

2026-06-26 20:00 Teams MVP 범위, 인증/secret, 저장소, observability, AI 인사이트, 협업 방식 논의 WIKI 회의록/전사 정리, docs/issue 반영

2026-06-27 16:11 이메일 멘토님이 주니어 온보딩 자료 공유 팀원 학습 자료로 공유 예정

2026-06-27 19:51 이메일 답변 온보딩 자료 감사 및 팀원 공유 예정 답변 학습 방향 자료로 활용

Appendix - 근거 문서 목록

종류 경로

source 프로젝트 계획 `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/docs/coach-one-page-plan.md`

source 아키텍처 `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/docs/architecture.md`

source 이벤트 문서 `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/docs/events.md`

source 팀 분배 `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/docs/team/work-allocation.md`

Gateway/Auth 가이드 `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/docs/team/member-guides/gateway-auth.md`

Target/Telemetry 가이드 `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/docs/team/member-guides/target-telemetry.md`

WIKI 제품 정의 `/Users/woonyong/workspace/Krafton-Jungle/WIKI/projects/final/product-definition.md`

WIKI MVP 계획 `/Users/woonyong/workspace/Krafton-Jungle/WIKI/projects/final/mvp-plan.md`

WIKI WBS `/Users/woonyong/workspace/Krafton-Jungle/WIKI/projects/final/wbs.md`

WIKI 멘토링 회의록 `/Users/woonyong/workspace/Krafton-Jungle/WIKI/projects/final/meetings/2026-06-26-mentoring.md`

WBS.xlsx `/Users/woonyong/workspace/Krafton-Jungle/SW\_AI\_W17-21-final/outputs/final-wbs-20260626/나만무\_상세\_WBS.xlsx`

Appendix - GitHub 근거

[Merged PR]

PR 내용

#538 Evidence 학습 가이드와 데모 추가, dev merge

#539 Evidence 학습 가이드와 데모 추가, main merge

#540 GitOps polling API 설계 정리, dev merge

#541 GitOps polling API 설계 정리, main merge

[Open Issues]

범위 이슈

상위 구현 이슈 #501~#509

Gateway/Auth 세부 #519~#526

Target/Telemetry 세부 #527~#537

[주요 브랜치]

브랜치 상태

`origin/feat/woonyong-kr/event-system` event runtime, DLQ, outbox, docs, crash test fix

`origin/feat/jcbbbbbb/api-gateway` email/password + Redis session

`origin/feat/minmings111/target-cluster-agent` target telemetry 진행, 경로 정리 필요

`origin/feat/jeonwoohyun-hydromel/\*` GitOps/Command 초기 브랜치

303호 4팀 · 주간 공유 / 멘토링 보고서 v2 : 2026-06-29

`origin/feat/ummfieg/\*` RCA/Audit/Projection 초기 브랜치

Appendix - 보고서 작성 이후 확인 필요

확인 필요 이유

최신 event-system branch 기준 GitHub Actions 결과 local 검증은 있으나 원격 CI 증거 필요

`make smoke` 실패 원인 재검증 Gateway 미실행으로 실패했으므로 실제 make up 환경에서 다시 확인 필요

Gateway/Auth 브랜치 dev rebase 46 commits behind로 PR 충돌 위험

Target/Telemetry 경로 정리 과거 `services/target-cluster-agent` 경로 변경이 현재 구조와 불일치

RCA 구현 diff 담당 브랜치 반영 source에는 증거가 있으나 담당 원격 브랜치가 초기 상태

Project WBS status 최신화 이슈 close가 아니라 Project status로 진행/완료 관리